

goofytex: Retrieval de Bundle a Producto en Moda

Documentacion Tecnica del Sistema

Equipo goofytex
Informe Tecnico Interno

Abstract—Este documento describe el sistema de retrieval bundle→product de goofytex. Se detallan las rutas actuales del repositorio tras la reorganizacion de carpetas, incluyendo preprocesamiento en `src/utils/`, entrenamiento e inferencia en `src/`, y capas de reranking/ensemble. El objetivo es mantener una referencia tecnica reproducible y alineada con la estructura real del codigo.

Index Terms—retrieval visual, OpenCLIP, SigLIP, YOLO, reranking, reproducibilidad

I. INTRODUCCION

La tarea en goofytex es retrieval visual multietiqueta: dada una imagen de bundle, el sistema debe recuperar los `product_asset_id` correctos y ordenarlos por relevancia.

Tras la reorganizacion del repositorio, el flujo operativo se distribuye en tres bloques:

- 1) Preparacion de datos e imagenes en `src/utils/` (`download.sh`, `preprocess_data.py`, `offline_augment.py`).
- 2) Entrenamiento y modelado en `src/` (`train.py`, `models/retrieval_openclip.py`).
- 3) Inferencia, reranking y export de submission en `src/` (`infer*.py`, `rerank/train_mlp.py`, `ensemble_csv.py`).

Este documento actualiza rutas y componentes para que la documentacion coincida con el estado real del workspace.

II. DATOS Y PREPROCESAMIENTO

A. Datasets de origen

Archivo	Proposito
<code>data/bundles_dataset.csv</code>	Catologo de bundles y URLs de imagen.
<code>data/product_dataset.csv</code>	Catologo de productos, URLs y descripcion textual.
<code>data/bundles_product_mappings_train.csv</code>	Bundle-train-superv bundle-producto para entrenamiento.
<code>data/bundles_product_mappings_test.csv</code>	Bundle-test-test para predecir <code>product_asset_id</code> .

TABLE I

DATASETS USADOS POR ENTRENAMIENTO E INFERENCIA.

B. Rutas de preprocesamiento actualizadas

El preprocesamiento y utilidades de datos se ejecutan desde `src/utils/`:

- `src/utils/download.sh`

- `src/utils/preprocess_data.py`
- `src/utils/offline_augment.py`
- `src/utils/add_gender.py`

Artefactos principales generados:

- `data/manifests/train_manifest.jsonl`
- `data/manifests/val_manifest.jsonl`
- `data/offline_aug/`
- `data/product_dataset_with_gender.csv`

III. METODOLOGIA

A. Formulacion

Para cada bundle b , el sistema rankea candidatos de producto $\{p_i\}_{i=1}^N$ y devuelve hasta 15 IDs por bundle.

B. Representacion y retrieval

El backend principal usa OpenCLIP Marqo Fashion SigLIP [1], [2]. La similitud se calcula por coseno sobre embeddings normalizados:

$$\text{sim}(\mathbf{q}, \mathbf{P}) = \mathbf{q}\mathbf{P}^\top \quad (1)$$

C. Deteccion por regiones

Cuando se habilita deteccion, se usan cajas de ropa con YOLO [3], con cache reutilizable en JSON. Si no hay detecciones validas, se usa fallback a imagen completa.

D. Reranking

El pipeline incluye capas opcionales de post-procesado:

- filtros de genero/categoria/score;
- ajuste por timestamp;
- reranker MLP entrenado en `src/rerank/train_mlp.py`;
- fusion de submissions por RRF en `src/ensemble_csv.py`.

E. Inventario de ideas implementadas

El codigo actual incluye las siguientes ideas tecnicas, ya implementadas en scripts:

- entrenamiento contrastivo OpenCLIP bundle $\text{left} \rightarrow \text{right}$ product con soporte AMP, acumulacion de gradiente y mineria de negativos duros (`src/models/retrieval_openclip.py`);
- retrieval guiado por deteccion YOLO con cache de cajas y fallback a imagen completa (`src/infer.py`, `src/new_infer.py`);
- pipeline per-crop con TTA, agregacion multi-crop y indexado FAISS opcional (`src/new_infer.py`);

- filtrado por seccion + clasificacion zero-shot de categoria por crop + boost semantico (`src/infer_phase1.py`);
- estrategia top-1 por crop para predicciones compactas (`src/infer_top1.py`);
- rerank temporal por proximidad de timestamps extraidos de URL (`src/new_infer.py`, `src/infer_phase1.py`);
- reranker MLP listwise con features de pares embedding y mezcla con score base (`src/rerank/train_mlp.py`, `src/infer_phase1.py`);
- penalizacion de hubness y rerank con modelo pesado de vision (late interaction) (`src/reranker.py`);
- ensemble final de predicciones por Reciprocal Rank Fusion (`src/ensemble_csv.py`);
- utilidades de datos reproducibles: preprocesado robusto, augmentacion offline determinista, inferencia de genero y analitica temporal (`src/utils/`).

IV. DETALLES DE IMPLEMENTACION

A. Configuracion y entypoints

El proyecto usa Hydra [4] con dataclasses tipadas (`src/config.py`). Entypoints principales:

- `python -m src.train`
- `python -m src.infer`
- `python -m src.new_infer`
- `python -m src.infer_phase1`
- `python -m src.infer_top1`
- `python -m src.infer_openclip`

B. Mapa de modulos (rutas vigentes)

V. PLAN EXPERIMENTAL

A. Objetivo

Definir comparativas reproducibles entre variantes de embedding, deteccion, retrieval y reranking, sin reportar resultados numericos en este documento.

B. Matriz de pruebas

- Embeddings: OpenCLIP Marqo SigLIP, SigLIP y baselines torchvision.
- Deteccion: full-image vs. crops YOLO.
- Indexado: busqueda directa vs. FAISS opcional.
- Post-procesado: filtros, TS rerank, MLP rerank.
- Ensemble: RRF sobre multiples submissions.

C. Artefactos por corrida

- `retrieval_openclip/best.pt`, `epoch_X.pt`, `metrics.jsonl`
- `test_submission*.csv`, `val_metrics*.json`
- caches de deteccion y metadatos en `data/` y `outputs/`

Modulo / Ruta	Responsabilidad
<code>src/train.py</code>	Bootstrap Hydra y despacho de backend de entrenamiento.
<code>src/models/retrieval_openclip.py</code>	Modelo de retrieval por crop y metricas JSONL.
<code>src/infer.py</code>	Inferencia principal OpenCLIP + YOLO + filtros.
<code>src/new_infer.py</code>	Variante per-crop con TTA y FAISS opcional.
<code>src/infer_phase1.py</code>	Variante por seccion + zero-shot por categoria.
<code>src/infer_top1.py</code>	Variante top-1 por crop.
<code>src/infer_openclip.py</code>	Script legacy de inferencia por argparse.
<code>src/rerank/train_mlp.py</code>	Entrenamiento de reranker MLP listwise.
<code>src/reranker.py</code>	Rerankers de post-procesado (hubness/heavy model).
<code>src/ensemble_csv.py</code>	Ensemble de submissions por RRF.
<code>src/utils/preprocess_data.py</code>	Validacion de esquema, split y manifests.
<code>src/utils/offline_augment.py</code>	Augmento offline determinista para imagenes.
<code>src/utils/download.sh</code>	Descarga de imagenes de bundles y productos.
<code>src/notebooks/</code>	Notebooks de EDA y baseline.

TABLE II

RUTAS ACTUALIZADAS TRAS LA REORGANIZACION DEL REPOSITORIO.

VI. REPRODUCIBILIDAD

A. Entorno

Dependencias base en `requirements.txt`: `pandas`, `numpy`, `pillow`, `tqdm`, `requests`, `hydra-core`, `torch`, `torchvision`.

Dependencias adicionales:

- `open_clip_torch`
- `ultralyticsplus`
- `faiss` (opcional)

B. Configuracion centralizada

- `config/config.yaml`
- `config/files/file.yaml`

C. Rutas de utilidades

Scripts auxiliares relevantes:

- `src/utils/preprocess_data.py`
- `src/utils/offline_augment.py`
- `src/utils/add_gender.py`
- `src/utils/check_link_timestamps.py`

VII. LIMITACIONES Y RIESGOS

- 1) Dependencia de pesos/modelos externos (OpenCLIP, YOLO).
- 2) Coste computacional adicional en variantes con deteccion + rerank.
- 3) Dependencias opcionales (FAISS/Ultralytics) fuera del set base de `requirements.txt`.
- 4) Cobertura de tests automatizados todavia limitada.

VIII. CONCLUSIONES

La documentacion de goofytex queda alineada con la estructura actual del repositorio, especialmente en rutas de utilidades y workflows de inferencia/reranking.

Las referencias de comandos, modulos y carpetas se han actualizado para reflejar la organizacion vigente en `src/`, `src/utils/`, `src/rerank/` y `documentacion/`.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the International Conference on Machine Learning*, 2021.
- [2] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *arXiv preprint arXiv:2303.15343*, 2023.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” GitHub repository, 2023.
- [4] O. Yadan, “Hydra: A framework for elegantly configuring complex applications,” GitHub repository, 2019.

APPENDIX

APENDICE: COMANDOS REPRODUCIBLES

A. Instalacion

```
pip install -r requirements.txt
pip install open_clip_torch ultralyticsplus
# opcional: pip install faiss-cpu
```

B. Descarga de imagenes

```
bash src/utils/download.sh
```

C. Preprocesamiento

```
python src/utils/preprocess_data.py \
    --skip_download \
    --out_dir data \
    --val_ratio 0.1 \
    --seed 42
```

D. Aumento offline

```
python src/utils/offline_augment.py \
    --bundles_manifest data/manifests/train_manifest.jsonl \
    --products_manifest data/product_dataset.csv \
    --products_images_dir data/product_images \
    --out_dir data/offline_aug \
    --bundles_num_augs 4 \
    --products_num_augs 2 \
    --img_size 224 \
    --seed 42 \
    --workers 8
```

E. Generacion de genero por producto

```
python src/utils/add_gender.py
```

F. Entrenamiento

```
python -m src.train
```

G. Inferencia principal

```
python -m src.infer infer.checkpoint_path=<ruta_a_best.pt>
```

H. Inferencia v8

```
python -m src.new_infer infer.checkpoint_path=<ruta_a_best.pt>
```

I. Inferencia Phase 1

```
python -m src.infer_phase1 infer.checkpoint_path=<ruta_a_best.pt>
```

J. Inferencia Top-1

```
python -m src.infer_top1 infer.checkpoint_path=<ruta_a_best.pt>
```

K. Inferencia OpenCLIP legacy

```
python -m src.infer_openclip \
    --checkpoint outputs/retrieval_openclip/best.pt \
    --submission-out outputs/retrieval_openclip/submission.csv
```

L. Reranker MLP

```
python -m src.rerank.train_mlp \
    --query-embeddings artifacts/embeddings/train_bundle_embeddings.pt \
    --product-embeddings outputs/product_embeddings.pt \
    --train-csv data/bundles_product_match_train.csv \
    --output artifacts/rerank/mlp_reranker.pt
```

M. Ensemble RRF

```
python src/ensemble_csv.py outputs/sub1.csv outputs/sub2.csv \
    -o outputs/ensemble_submission.csv
```

N. Reporte de timestamps

```
python src/utils/check_link_timestamps.py \
    --out-csv outputs/ts_date_alignment_report.csv \
    --timezone UTC
```